

AI GENERALIST ACCELERATOR · 14-DAY AI SPRINT

Day Nine

Building Stronger Harnesses: Grokbot, Local Models, and Changing the Pilot

A cleaned, detailed record of the live session and Q&A briefing

Topics covered: What a Harness Is · Cockpit and Pilot · Building a ChatGPT Harness · The PILOT Framework · Invisible Harnesses and Grokbot · AI Employees, Managers, and a Chief of Staff · Tool-Driven to Framework-Driven · Ollama: Local and Cloud Models · Open Weight, Open Source, and Licences · Skills From skills.sh

About This Session

Day nine brought Dileep Karri back as promised, voice still hoarse from the flu that had kept him off day five and delivered, he warned, at the maximum volume he could sustain, headphones advised. Sumedha's opening poll asked which tool had surprised the cohort most after eight days, n8n flooding the chat with Lovable, Claude, and image-and-video close behind, and framed the evening as one step behind those tools, into what actually sits underneath the AI coding workflows: harnesses and open-source models, Grokbot, and Codex as a harness. There was no workbook, by design, partly to give a breather before the hackathon and partly because the material was dense, so the breakout rooms became mentor-led Q&A; instead, and the session was explicitly a levelling-up of last week's principles rather than a new skill, with Dileep's promise that the sun-rises-in-the-east framing of day one would be revised again when he returns. Questions went to the Q&A; box, answered as the material reached them.

HOW TO USE THESE NOTES

Nothing here needs a subscription to start: every harness component was installed by pasting a documentation URL into the ChatGPT desktop app and saying install this if not yet installed, and Ollama's local models are free. The desktop apps are essential, none of this works on the web or mobile versions. Recordings land on the platform within twelve hours, and the session transcript sits beside the summary in the post-read section.

Dileep asked anyone who found the session heavy to watch the recording once or twice; it was meant to be a jump, not a lesson. Tomorrow's session announces the hackathon groups and how to reach your teammates, which runs from Thursday to Saturday night, so attendance matters; hackathon logistics go to the tech-support channel, not the mentors.

What a Harness Is: Cockpit and Pilot

Every AI tool the cohort had used was, until day five, transactional, you type, it answers, turn after turn; then on day five AI made a Nike ad rather than a script for one, the tool finishing the job. The thing that makes that possible is the harness, defined as the software around AI that lets it do a job, which in practice is the connectors and the skills together: Higgsfield was a harness for the ad, the ad-writing skill was another, and every plugin in ChatGPT or connector in Claude, Gmail, Tableau, the rest, is a piece of harness giving the model hands, eyes, and legs. The model decides, the harness does. The analogy that carried the evening was the flight: the job is flying from A to B, the model is the pilot, the harness is the cockpit, and neither works without the other, the best pilot in an old-fashioned plane cannot fly it well and the most modern cockpit with a weak pilot crashes. ChatGPT has strong pilots in Astra and the Sol models, Claude in Fable and Opus, so the work now is setting up the cockpit, and the question changes accordingly: no longer is Claude better or is ChatGPT better, but how good is your harness. The loop was drawn on the whiteboard as four steps, you give AI a job, the model decides the next step, the harness runs it, the model reads the result and decides again, illustrated with a bridge, the AI decides whether to cross, jump, or break it and the harness does the crossing, and stated as the equation the whole sprint had been circling: an AI agent equals model plus harness. Model innovation happens at a few labs; everyone else, Lovable, Bolt, Parallel, builds harness. The cockpit gives the pilot standing orders, which is what day one's custom instructions and project instructions are, sticky notes for the harness; hands to read, write, search, and click; a window to see through; and stop signs where it must check with you.

Building a ChatGPT Harness in Twenty Minutes

The demonstration built a knowledge-work harness inside the ChatGPT desktop app, one capability at a time, each installed by the same non-technical method. Documentation first: Context7 supplies up-to-date documentation to any agent, which matters because vibe coding on stale documentation hallucinates, added from the plugin list with a click and an authentication, and identical in Claude. Then search, because a stronger search engine means less hallucination: Parallel Search, from a company founded by Twitter's former CEO to build search for agents, free to a point, with Tavily and Exa as alternatives, installed by copying its documentation URL into a new chat and typing install Parallel Search MCP if not yet set up, the ChatGPT app reading the docs and doing the rest. Then a scraper for web pages, Firecrawl, installed by copying its one-click prompt and saying go for it, and proven by scraping a product-design page and explaining it like a five-year-old. Then heavier scraping of YouTube and Instagram through Apify, paid, authorised in a window the app opened by itself, with the free alternative yt-dlp installed by pasting its GitHub URL, and proven on one of Vaibhav's videos, transcript scraped and summarised in one command. Two rules travelled with the demonstrations: give the app full access while installing and tighten permissions afterwards, and expect this only on desktop, never on web or mobile. The recap was the method itself, copy the documentation link, paste, say install, and his justification for scrapers in particular, a compliance check that must read a government site, or research that lives inside videos, work that plain web search cannot reach.

The PILOT Framework

To make harness design teachable, Dileep offered an acronym of his own, not a universal term: PILOT. Permission, what the harness is allowed to do. Information, what context and imports it needs to do the job. Loops, how many times it may retry before it stops, because an AI told to keep going until finished will loop forever and never finish. Operations, the tools and skills it is given. Test, how you verify it works, the part where security-first products like Meta's Muse invest most heavily. Then the honest question, how many were overwhelmed by setting up a harness by hand, full-access mode, URLs, installs, and the honest answer that a great many people are, which is exactly why the next generation of products, Grokbot foremost, and Instinct and Muse by Meta named without demonstration, exists to make the harness invisible and easy to set up.

Grokbot: AI Employees, Managers, and a Chief of Staff

Grokbot is bundled with the paid Grok plans, the twenty-dollar SuperGrok tier and the three-hundred-dollar one, and also with Cursor's paid plans, and its interface is minimal because its idea is simple: every bot is an AI employee built for one task. A social-research bot was created by conversation, Twitter and YouTube research on a topic, summarise the findings, and given its first assignment, Vaibhav's thoughts on AGI, with no harness configured at all, because the harness underneath prompted for what it needed, in this case an Apify token when X refused to load, pasted in and saved securely, after which it scraped the tweets and offered to scrape the videos too. The three differences from ChatGPT and Claude were named: Grokbot tells you what harness a task requires rather than leaving you to imagine it; it runs on a dedicated cloud computer per account, so installations happen there, not on your machine, and a scheduled bot keeps running after your laptop is shut, which a ChatGPT or Claude automation does not; and you cannot change the model, only Grok's own. A job-scraper bot followed, built by asking to be interviewed as a non-technical user, handed his LinkedIn URL, and returning a fifty-fifty list of director-level AI roles that he refined with feedback, keep to the main angle, add a fitment score, at which point Grokbot wrote itself a skill so future runs follow the same pattern, and offered a tailored resume and cover letter for the top role while refusing, correctly, to hit submit without him. The marketplace supplies plugins and, most usefully, Dr. Eggbot, a bot that builds bots the way a skill creator builds skills, used to spin up an analogy bot; a group chat of bots then let the research bot and the analogy bot talk to each other, producing, on the second attempt, the analogy he kept, same brain, two jobs, one alone in a room with a notepad, the other with a phone, a shared drive, a checklist, and a manager who signs off, the model as brain and the harness as workplace. The recommended structure comes from the SpaceX team that built the product: run bots like employees under managers, so a research lead and a job lead were created with a chief of staff above them, and one message to the chief of staff, research GPT-6 Astra use cases and in parallel find India-remote jobs that suit me, fanned out to both leads and merged their replies into a status brief, marketing bots under a CMO, operations bots under a COO, all reporting to the chief of staff. His own advice was gentler than SpaceX's: start with two or three employees, nurture them, strengthen them with skills through Dr. Eggbot, connect Gmail and calendar from the marketplace, and build upward only once the workers are good, with the reminder that the last ten percent, the actual application click under your name, stays human. Weekly usage is visible in settings, four percent after the whole demonstration.

From Tool-Driven to Framework-Driven

The question everyone had, when to use Grokbot and when ChatGPT, was named as the trap itself. ChatGPT, Claude, and Grokbot are Americano, cappuccino, and latte: the chat split on which is better, and the point is that it is taste, which coffee gets your day started. The reframed question is never which tool is best but which harness setup lets you do your work, within your budget, applying your principles, so if ChatGPT works for you, so be it, whatever Twitter and YouTube say about who killed whom this week. That is the move from tool-driven to framework-driven, day one's sun rising in the east giving way to Copernicus: do I have the right skills, can I make my own, do I have the right tools, do I know how to connect them, which is now nothing more than finding the documentation URL, pasting it, and saying install.

Changing the Pilot: Ollama, Local Models, and Cloud Models

The second half flipped the story from the cockpit to the pilot, showing how the other models the cohort could see in ChatGPT's model list got there. The application is Ollama, a desktop app that holds two kinds of model: local models installed on your own machine, free with no subscription, and cloud models hosted by Ollama under a twenty-dollar plan and privacy by default, conversations processed and returned but never stored. The sizing rule was repeated three times so nobody could miss it, choose a model no larger than half your physical RAM, not VRAM: sixteen gigabytes means models of eight gigabytes or eight billion parameters at most, thirty-two gigabytes means around sixteen, and eight gigabytes means Gemma's two-billion-parameter small model, because a model larger than your RAM will crash or freeze the machine, which is why Dileep ran the demonstration on cloud models, having crashed his own laptop three times in six cohorts running local models beside a screen-shared Zoom. Installation is a search on ollama.com for a model name, Gemma from Google or Qwen from the Alibaba family being the safe recommendations, with Qwen's coder variants for code and DeepSeek and Kimi as larger options, then copying the exact name into the Ollama app, where saying hi to a selected model downloads it. Cloud models are marked with a cloud icon, GLM 5.3 and Kimi K3 among them. Then the connection that was the day's promised skill: Ollama's Apps panel has a Connect button for the ChatGPT desktop app, its Settings choose which models to expose, save and restart, and the models appear in ChatGPT's own picker, sometimes after a reinstall or update of Ollama and a second attempt. The test proved the thesis of the day. The same harness, Parallel Search included, ran on GLM 5.3 with the news on Grok 4.7; the same Gmail connector and PowerPoint skill ran on Kimi K3 to turn the latest Staying Ahead AI newsletter into a deck, in parallel with GPT 5.6 doing the same, both producing decks, the local pilot less elegant and without memory access but working, the cockpit intact while the pilot changed. Local models are weaker pilots, fit for reformatting, summarising, and simple tasks when limits are hit or when compliance requires that no data leave the machine. Claude connects too, with a quirk, its picker always shows Fable, Opus, Sonnet, and Haiku, so you assign a local model to each name and remember the mapping, and Claude's own models are locked out until Ollama is disconnected, which is why the demonstration used ChatGPT, whose official Ollama integration keeps both. Msty and LM Studio download models as well, but only Ollama plugs into the harness already built.

Open Weight, Open Source, Licences, and a Skill of Your Own

While the decks rendered, two distinctions got their analogies. Most of the models on Ollama are open weight, not open source: open source hands over the song and the instruments, their make, their tuning, everything needed to reproduce it, while open weight shares only a snapshot of the mixer settings, or, in the recipe version, the ingredients without the sources, enough to use but not to replicate exactly. Licences came next, the two that matter told through a sourdough recipe. MIT says take my recipe, change it, sell it, do anything, so long as you credit me; GPL says take it and change it, but every change must be shared with the world, so no secret sourdough 2.0. For commercial products, look for MIT, and for anything else ask Perplexity or an AI what the licence means. The closing build showed skills.sh, the library where a copied prompt installs a skill in ChatGPT or Claude, and improved on it: two No-AI-Slop skills were opened, their prompts copied rather than their install commands, pasted together with the instruction build me a skill combined from the above to remove AI slop from writing, and then given a personal flavour through the harness, go to my Gmail, learn the writing patterns from my emails, and add them to the combined skill. That, he said, is the intelligent way to build a harness, never installing a skill raw but adding your branding, your patterns, your small preferences, with no limit on how many skills combine.

THE MENTAL MODEL

An AI agent is a model plus a harness: the pilot decides, the cockpit executes. Build the cockpit by pasting documentation links and saying install, judge it by PILOT, permission, information, loops, operations, test, and stop asking which model is best. Where the harness is too much, let Grokbot make it invisible and run employees under managers. Where the pilot is the limit, swap it through Ollama, half your RAM, Gemma or Qwen, same harness, same skills, and check the licence before you sell.

Closing and What Comes Next

The recap tied the session to the week before: the AI agent everyone kept naming is a model plus a harness of tools and skills, the model can be ChatGPT, Claude, Grokbot, or a local pilot, the harness comes from plugins, connectors, and skills.sh, and once the harness is strong the model can be swapped through Ollama without losing a thing. Dileep signed off with socials in chat and the promise of more sessions on deeper topics. Sumedha's wrap ran the poll, confirmed that day six and day seven's second recordings sit lower on their platform pages, opened Q&A; breakout rooms in place of a workbook, and made the announcement that shaped the rest of the week: tomorrow's session reveals the hackathon groups and how to connect with teammates, with the hackathon itself running Thursday to Saturday night, every detail to be covered tomorrow and the day after.

Key Takeaways

Stripped to its essentials, the session made the following points.

- **Agent equals model plus harness.** The model decides the next step and the harness runs it; connectors and skills are the cockpit, the model is the pilot, and neither flies alone.

- **Install by pointing, not by coding.** Context7 for documentation, Parallel Search for search, Firecrawl and Apify or yt-dlp for scraping, each added to the ChatGPT desktop app by pasting a documentation URL and saying install, full access during setup, tightened after.
- **Design harnesses with PILOT.** Permission, information, loops, operations, and test, with loop limits so an agent never runs forever and testing where security-first products put their effort.
- **Grokbot makes the harness invisible.** One bot per task on a dedicated cloud computer that keeps running when your laptop sleeps, Dr. Eggbot to build bots, group chats so they talk, and employees under managers under a chief of staff, starting with two or three.
- **Stop asking which model is best.** ChatGPT, Claude, and Grokbot are coffees; the framework question is whether your skills, tools, and budget let you do your work, which is the shift from tool-driven to framework-driven.
- **Swap the pilot through Ollama.** Local models at half your RAM, Gemma or Qwen, are free and keep data on the machine; cloud models are privacy-first for twenty dollars; connect the ChatGPT desktop app and the whole harness runs on the new model.
- **Know open weight from open source, and MIT from GPL.** Weights are the mixer settings without the instruments; MIT wants credit, GPL wants every change shared, so check the licence before building a product.
- **Never install a skill raw.** Copy prompts from skills.sh, combine them, and let the harness add your own writing patterns from your inbox, because the strongest harness is the one flavoured with you.

End of Day Nine notes. Next: the hackathon groups — Day Ten.